



LIBRARY MANAGEMENT SYSTEM

Project Report
CS-116 Object Oriented Programming

Group Members:

Shaffay Zia (CS-082) || Abdul Wahab (CS-138) || Hassan Khan (CS-078) ||
Mohammad Ali (CS-088)

Section: B

Group ID: G3-8

Date: 05/10/2026

Instructor:

Miss Fauzia Yasir

Department of Computer Systems Engineering

NED University of Engineering & Technology

Spring 2026

CONTENTS

<i>CONTRIBUTION PAGE</i>	<i>-</i>
<i>CHAPTER 1 – INTRODUCTION</i>	<i>1</i>
<i>CHAPTER 2 – CLASS DESIGN & SYSTEM STRUCTURE</i>	<i>2</i>
<i>CHAPTER 3 – OOP CONCEPTS</i>	<i>8</i>
<i>CHAPTER 4 – OUTPUT AND TESTING</i>	<i>9</i>

CONTRIBUTION PAGE

Mohammad Ali (CS-088)

During the **first phase of the project**, which focused on graphical user interface development, Mohammad Ali was assigned the **Librarian System (Admin + CRUD)** portion of the application. His work included the development of the **Librarian Dashboard, Book Catalog (Admin CRUD), Reports, User Management, and Reviews (Moderation)**. This contribution formed the administrative side of the interface and supported the management-oriented functionality of the system.

During the **second phase**, which involved backend C++ development, he was responsible for implementing **Database, LibrarySystem, and LibrarySystemBridge**. His work in this phase covered database handling, system-level coordination, and the bridge required for communication between the backend logic and the QML frontend.

In the **third phase**, Mohammad Ali played a key role in the **integration of all modules** and also contributed to **overall project management**. He helped coordinate the connection of the frontend, backend, and database components, ensuring that the final application functioned as a unified and complete system.

Hassan Khan (CS-078)

During the **first phase of the project**, Hassan Khan worked on **Auth + App Core + Navigation** in the graphical user interface. His assigned screens included the **Login Screen, Signup Screen, and Profile Settings (shared base)**. His work in this phase contributed to the foundational entry flow and navigation structure of the application.

During the **second phase**, which focused on backend implementation, he was assigned the authentication and user-related classes including **Person, Student, Librarian, and AuthManager**. This involved developing the user hierarchy, authentication functionality, and role-based access support required by the system.

In the **third phase**, Hassan integrated his assigned module with the rest of the project and also contributed to **fixing user interface bugs** identified during testing and refinement. His support helped improve the usability and stability of the final interface.

Shaffay Zia (CS-082)

During the **first phase of the project**, Shaffay Zia worked on **Core Data + Shared Screens + Backend-oriented GUI work**. His assigned screens included **Book Catalog (Base structure)**, **Book Detail Page (Base)**, **Transactions (Core logic)**, and **Reviews (Base system)**. His work in this phase established several shared and logic-driven interface components of the application.

During the **second phase**, which involved backend C++ development, he was assigned the transaction and review-related classes including **transaction**, **transactionlog**, **Review**, **Reviewlog**, and **FineCalculator**. His contribution covered issue and return processing, review handling, and overdue fine calculation logic required for these modules.

In the **third phase**, Shaffay integrated his backend work into the complete project and also contributed significantly to **debugging and testing**. His efforts helped resolve implementation issues, improve system correctness, and strengthen the reliability of the final application.

Abdul Wahab (CS-138)

During the **first phase of the project**, Abdul Wahab worked on the **Student Experience (Frontend Focus)** portion of the graphical user interface. His assigned screens included the **Student Dashboard**, **Membership**, **Transactions (Student view)**, and **Reviews (Write/View)**. His contribution in this phase focused on the development of user-facing screens intended for student interaction.

During the **second phase**, which focused on backend C++ development, he was assigned the book and membership-related classes including **Book**, **BookCatalog**, **Membership**, **Silver**, **Gold**, **Platinum**, **Wallet**, and **WalletLog**. His work covered the management of book data, membership rules, and wallet-related functionality within the system.

In the **third phase**, Abdul Wahab integrated his assigned module with the rest of the project and also contributed to **debugging and issue resolution** during final testing. His efforts supported the stability and refinement of the complete system during the integration stage.

CHAPTER 1 – INTRODUCTION

Project Overview

The Library Management System is an Object-Oriented Programming project developed to digitize the core operations of a library in a structured and modular way. The system combines a C++ backend, a Qt/QML frontend, and a database layer to manage users, books, memberships, transactions, reviews, wallets, and system-level coordination. The project is designed to support both student and librarian roles, with different permissions and functions assigned to each type of user.

System Modules

The project is divided into multiple major modules. The authentication module manages users and login operations through Person, Student, Librarian, and AuthManager. The book and membership module manages book records, membership tiers, and wallet operations through Book, BookCatalog, Membership, Silver, Gold, Platinum, Wallet, and WalletLog. The transaction and review module manages library borrowing activity and student feedback through transaction, transactionlog, Review, Reviewlog, and FineCalculator. Finally, the system and integration module handles persistence and coordination through Database, LibrarySystem, and LibrarySystemBridge.

Development Approach

The project follows a layered development approach. The database layer stores and retrieves persistent records such as users, books, transactions, reviews, and wallets. The backend logic layer applies to the business rules of the system, such as issuing books, upgrading memberships, calculating fines, approving reviews, and returning books. The frontend bridge layer converts backend C++ objects into Qt-friendly data structures such as QVariantMap and QVariantList, which are then consumed by the QML interface. This separation makes the system easier to maintain and expand.

Class Design Overview

The system is built using multiple interacting classes, each designed around a specific responsibility. Some classes represent real entities such as books, users, transactions, and reviews, while other classes serve as managers or service classes that coordinate collections of objects and perform system-level operations. The relationships between these classes include inheritance, association, aggregation, dynamic type handling, and bridging between pure backend logic and the QML user interface.

CHAPTER 2 – CLASS DESIGN AND SYSTEM STRUCTURE

Person

Attributes: userID, name, email, password

Methods: constructor, destructor, getRole(), getUserID(), getName(), getEmail(), getPassword(), setName(), setEmail(), setPassword(), authenticate(), operator<<

Relationship: Base class of Student and Librarian through inheritance.

Purpose and Responsibility: The Person class acts as the abstract base class for all users in the system. It stores the common identity and authentication data shared by both students and librarians, and it provides a common interface for role-specific behavior.

Student

Attributes: department, borrowedCount, totalFineOwed, status, membership, wallet

Methods: constructor, destructor, setmembership(), getMembershipTier(), getRole(), getStatus(), setStatus(), getTotalFineOwed(), get_BorrowedCount()

Relationship: Inherits from Person and is associated with Membership and Wallet through pointers.

Purpose and Responsibility: The Student class represents a student user of the library system. It extends the common user data inherited from Person and adds membership, borrowing, fine, and account status information required for student-side library operations.

Librarian

Attributes: employeeCode

Methods: constructor, getRole()

Relationship: Inherits from Person.

Purpose and Responsibility: The Librarian class represents the administrative user of the system. It specializes the Person class by identifying a user who can manage books, users, transactions, and reviews from the staff side of the application.

AuthManager

Attributes: userCount, registeredUsers

Methods: destructor, getUserCount(), findById(), getAllUsers(), registerPerson(), upgrademembership(), login(), updateUser()

Relationship: Aggregates multiple Person pointers and works with Student objects through dynamic_cast.

Purpose and Responsibility: The AuthManager class is the user-management and authentication service of the backend. It maintains registered users, supports login, updates user details, performs membership upgrades, and provides user lookup operations.

Book

Attributes: isbn, title, author, category, section, publisher, edition, language, totalCopies, availableCopies, publicationYear, pages, status

Methods: constructor, getIsbn(), getTitle(), getAuthor(), getCategory(), getSection(), getPublisher(), getEdition(), getLanguage(), getPublicationYear(), getPages(), getTotalCopies(), getAvailableCopies(), getStatus(), setTitle(), setAuthor(), setCategory(), setSection(), setPublisher(), setEdition(), setLanguage(), setPublicationYear(), setPages(), setTotalCopies(), issueOneCopy(), returnOneCopy(), statusToString(), operator<<

Relationship: Managed by BookCatalog and referenced by transactions and reviews through ISBN.

Purpose and Responsibility: The Book class models one library book and stores both descriptive and inventory-related information. It supports copy issue and return behavior and updates its availability status according to the number of copies currently available.

BookCatalog

Attributes: books, bookcount

Methods: addBook(), updateBook(), removeBook(), getAllBooks(), findByIsbn(), searchByTitle(), searchByAuthor(), searchByCategory(), searchBySection()

Relationship: Aggregates multiple Book objects.

Purpose and Responsibility: The BookCatalog class manages the collection of books stored in the system. Its responsibility is to organize, search, insert, update, and remove book records for the overall catalog module.

Membership

Attributes: studentId, loanDurationDays, isActive, startDate, expiryDate

Methods: constructor, destructor, getTierName(), getBorrowedLimit(), getFineDiscount(), getRenewalFee(), getLoanDuration(), getStudentId(), getExpiryDate(), getStartDate(), getStatus(), isExpired(), renewMembership()

Relationship: Base class of Silver, Gold, and Platinum; associated with Student.

Purpose and Responsibility: The Membership class provides the common structure for all membership tiers in the project. It stores information such as duration, activity state, and validity period, while defining the interface for tier-specific benefits and limits.

Silver

Attributes: Inherited from Membership

Methods: constructor, getBorrowedLimit(), getFineDiscount(), getRenewalFee(), getTierName()

Relationship: Inherits from Membership.

Purpose and Responsibility: The Silver class represents the basic membership tier of the library system. It defines the default borrowing rules and fee behavior for regular members.

Gold

Attributes: Inherited from Membership

Methods: constructor, getBorrowedLimit(), getFineDiscount(), getRenewalFee(), getTierName()

Relationship: Inherits from Membership.

Purpose and Responsibility: The Gold class represents an upgraded membership tier. It provides better borrowing privileges and a fine discount compared to the basic membership level.

Platinum

Attributes: Inherited from Membership

Methods: constructor, getBorrowedLimit(), getFineDiscount(), getRenewalFee(), getTierName()

Relationship: Inherits from Membership.

Purpose and Responsibility: The Platinum class represents the highest membership tier available in the system. It offers the most generous borrowing limit and the greatest fine discount among all membership levels.

Wallet

Attributes: studentId, balance, suspended, SUSPENSION_THRESHOLD

Methods: constructors, getStudentId(), getBalance(), isSuspended(), addAmount(), deductFine(), deductMembershipRenewalFee()

Relationship: Associated with Student and managed by WalletLog.

Purpose and Responsibility: The Wallet class stores the monetary balance of a student account and supports balance changes caused by fines, renewals, and deposits. It also handles suspension logic when the balance falls below the threshold.

WalletLog

Attributes: WalletEntry, wallets, logs

Methods: addWallet(), createWallet(), **overloaded** createWallet(), getWallet(), isUserSuspended(), applyFine(), payMembershipRenewalFee(), addMoney()

Relationship: Aggregates multiple Wallet objects.

Purpose and Responsibility: The WalletLog class is associated with Wallet through aggregation, as it manages and stores multiple wallet objects. It also indirectly supports student-related financial operations such as fine deduction, renewal fee deduction, and suspension handling.

FineCalculator

Attributes: No data attributes

Methods: daysOverdue(), calculateBaseFine(), applyMembershipDiscount(), calculateFinalFine()

Relationship: Used by transaction-related logic and by LibrarySystem during return processing.

Purpose and Responsibility: The FineCalculator class is a utility class responsible for overdue fine computation. It calculates overdue days, computes the base fine, and then applies membership-based discounts so that fine rules remain separate from transaction storage logic.

Transaction

Attributes: transactionId, studentId, username, isbn, issueDate, dueDate, returnDate, status, bookName, fine

Methods: constructors, setTransactionId(), setStudentId(), setUsername(), setIsbn(), setIssueDate(), setDueDate(), setReturnDate(), setStatus(), setFine(), getTransactionId(), getStudentId(), getUsername(), setIsbn(), getIssueDate(), getDueDate(), getReturnDate(), getbookName(), getStatus(), getFine(), isActive(), isReturned(), isOverdue(), markReturned(), updateOverdueStatus(), display()

Relationship: Managed by transactionlog and associated with Student and Book through stored IDs and names.

Purpose and Responsibility: The transaction class is managed by transactionlog and is associated with students and books through stored identifiers such as student ID and ISBN, along with related display information such as username and book name. This allows each transaction record to connect a specific student with a specific book without directly owning those objects.

Transactionlog

Attributes: transactions, transactioncount

Methods: addTransaction(), removeTransaction(), findTransactionById(), hasActiveTransaction(), issueBook(), returnBook(), updateAllOverdue(), getTransactionsByStudent(), getOverdueTransactions(), displayAllTransactions(), getAllTransactions()

Relationship: Aggregates multiple transaction objects, uses FineCalculator, and is used by LibrarySystem.

Purpose and Responsibility: The transactionlog class manages all transaction records in memory. It provides the core transaction operations of the system, including issuing books, returning books, overdue checking, searching records, and retrieving a student's borrowing history.

Review

Attributes: reviewId, studentId, username, isbn, bookname, rating, comment, status, reviewDate

Methods: constructors, getReviewId(), getStudentId(), getUsername(), setIsbn(), getBookname(), getRating(), getComment(), getStatus(), getReviewDate(), isApproved(), isPending(), approve(), display()

Relationship: Managed by Reviewlog and associated with students and books through stored identifiers and names.

Purpose and Responsibility: The Review class is managed by Reviewlog and is associated with students and books through stored identifiers such as student ID and ISBN, along with related information such as username and book name. This relationship enables the system to track which student reviewed which book while keeping review records modular and easy to manage.

Reviewlog

Attributes: reviews, reviewcount

Methods: hasStudentReviewedBook(), addReview(), findReviewById(), approveReview(), deleteReview(), getReviewsByBook(), getReviewsByStudent(), getPendingReviews(), getApprovedReviews(), displayAllReviews(), getAllReviews()

Relationship: Aggregates multiple Review objects and is used by LibrarySystem.

Purpose and Responsibility: The Reviewlog class manages the collection of reviews in the backend. It prevents duplicate reviews for the same student-book pair, supports approval and deletion operations, and provides filtered retrieval of review data.

Database

Attributes: Database connection object and LoginResult structure

Methods: connect(), init(), addUser(), addBook(), addTransaction(), addReview(), addWallet(), getUsers(), getBooks(), getTransactions(), getReviews(), getWallets(), updateUser(), updateBook(), updateTransaction(), updateReview(), deleteUser(), deleteBook(), deleteTransaction(), deleteReview(), loginUser(), getMaxIdNumber()

Relationship: Used by LibrarySystem as the persistent storage layer.

Purpose and Responsibility: The Database class serves as the persistence layer of the system. It manages database connection, table initialization, and CRUD operations for users, books, transactions, reviews, and wallets.

LibrarySystem

Attributes: authManager, TransactionManager, ReviewManager, BooksManager, WalletsManager, FineCalc, currentUser

Methods: constructor, initializeSystem(), saveSystem(), loadUsersIntoSystem(), loadBooksIntoSystem(), loadTransactionsIntoSystem(), loadReviewsIntoSystem(), loadWalletsIntoSystem(), login(), logout(), generateId(), registerStudent(), registerLibrarian(), registerUser(), removeUser(), updateStudent(), updateLibrarian(), updateUser(), addbook(), removeBook(), updateBook(), issueBook(), returnBook(), upgradeStudentMembership(), getCurrentMembershipDetails(), renewMembership(), submitReview(), approveReview(), deleteReview(), getStudentDashboardData(), getStudentBorrowHistory(), getAllBooks(), getAllUsers(), getAllTransactions(), getAllReviews(), displayAllData()

Relationship: Composes and coordinates the main manager classes and uses Database for persistent storage.

Purpose and Responsibility: The LibrarySystem class is the central controller of the backend. It coordinates authentication, catalog operations, issue and return logic, membership handling, review processing, dashboard data, and communication with the database layer.

LibrarySystemBridge

Attributes: m_system

Methods: constructor, login(), logout(), registerStudent(), registerLibrarian(), registerUser(), updateUser(), removeUser(), getMembership(), getStudentDashboard(), getStudentBorrowHistory(), upgradeMembership(), renewMembership(), addBook(), removeBook(), updateBook(), issueBook(), returnBook(), submitReview(), approveReview(), deleteReview(), getBooks(), getUsers(), getTransactions(), getReviews()

Relationship: Wraps LibrarySystem and exposes backend operations to QML through Qt-friendly methods and data types.

Purpose and Responsibility: The LibrarySystemBridge class is the interface layer between the QML frontend and the C++ backend. It converts backend data into QVariant-based structures and exposes Q_INVOKABLE methods so that the user interface can communicate with the system logic cleanly.

CHAPTER 3 – OBJECT-ORIENTED PROGRAMMING CONCEPTS

Encapsulation

Encapsulation was implemented throughout the project by declaring class data members as private or protected and providing controlled access through public member functions. Classes such as Person, Book, transaction, and Review restrict direct access to internal attributes and instead use getters, setters, and utility methods to manage object state. This approach improves data security and ensures that internal data is modified in a controlled and meaningful manner.

Inheritance

Inheritance was applied in the project to establish logical hierarchical relationships between related classes. The classes Student and Librarian inherit from the base class Person, while the classes Silver, Gold, and Platinum inherit from the base class Membership. This reduced repetition enabled the reuse of common attributes and methods while allowing derived classes to define specialized behavior relevant to their respective roles.

Aggregation / Composition

Aggregation and composition were used in the design of manager and controller classes that maintain collections of other objects. For example, BookCatalog manages multiple Book objects, transactionlog manages multiple transaction records, Reviewlog manages multiple Review objects, and WalletLog manages multiple Wallet objects. Similarly, LibrarySystem combines several major service classes, enabling the project to function as one integrated system while preserving modular structure.

Polymorphism

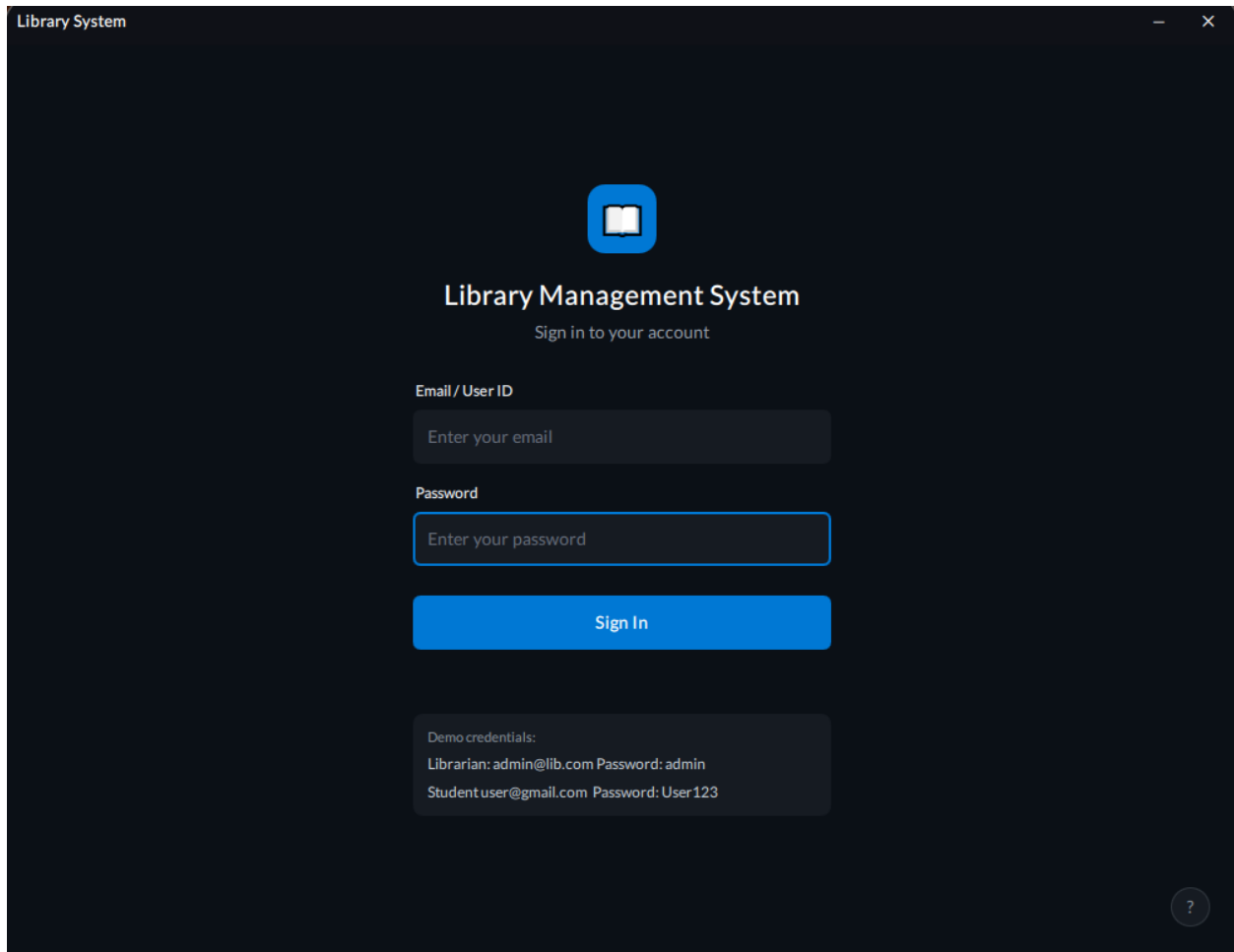
Polymorphism was utilized using virtual functions and base-class pointers. The Person class declares the pure virtual function `getRole()`, which is implemented differently by Student and Librarian, while the Membership class defines virtual functions that are overridden by Silver, Gold, and Platinum. In addition, runtime polymorphism is supported using `dynamic_cast` in classes such as AuthManager and LibrarySystem, allowing the program to safely identify and work with derived objects through base-class references and pointers.

Association

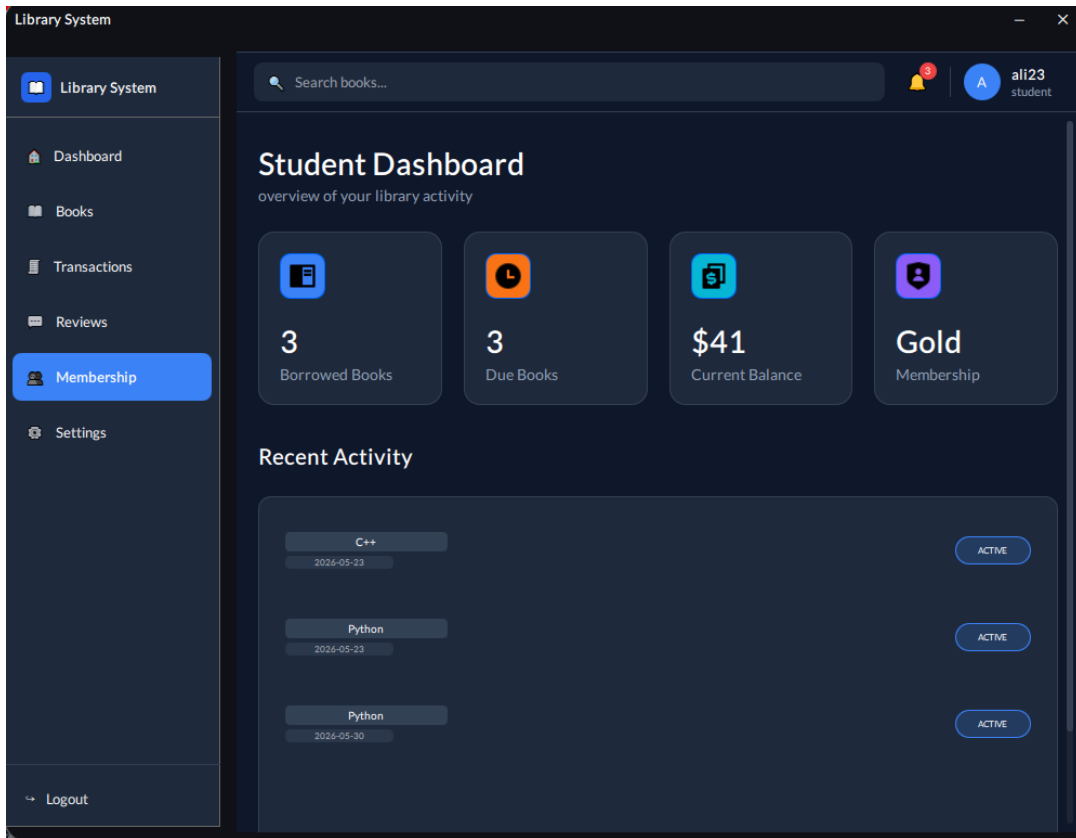
Association was used in cases where classes needed to interact with one another without inheriting from each other. For instance, transaction records are associated with students and books through attributes such as student ID, username, ISBN, and book name, while review records are similarly associated with students and books through identifying information. This form of relationship enabled the system to connect related data across modules without introducing unnecessary inheritance or ownership dependencies.

CHAPTER 4 – OUTPUT AND TESTING

Screenshots below demonstrate various features of the working Library Management System.



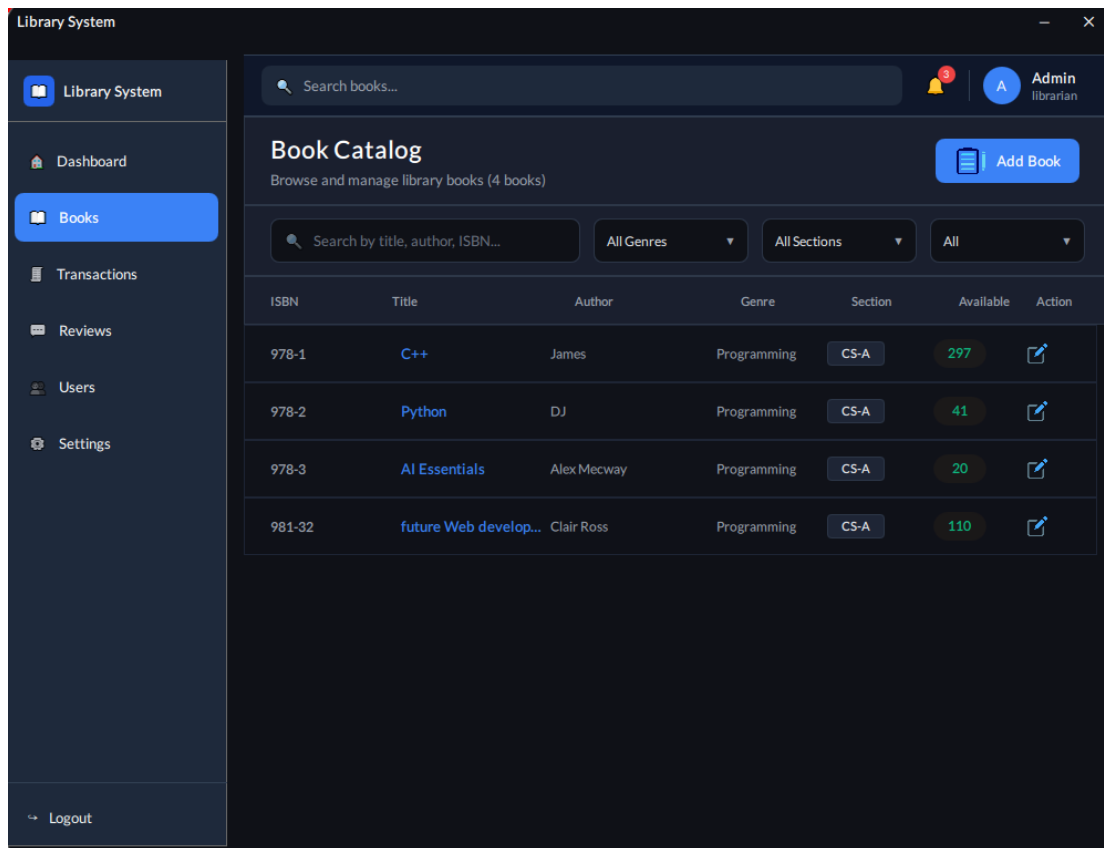
(Figure 4.1: LMS Sign in Portal)



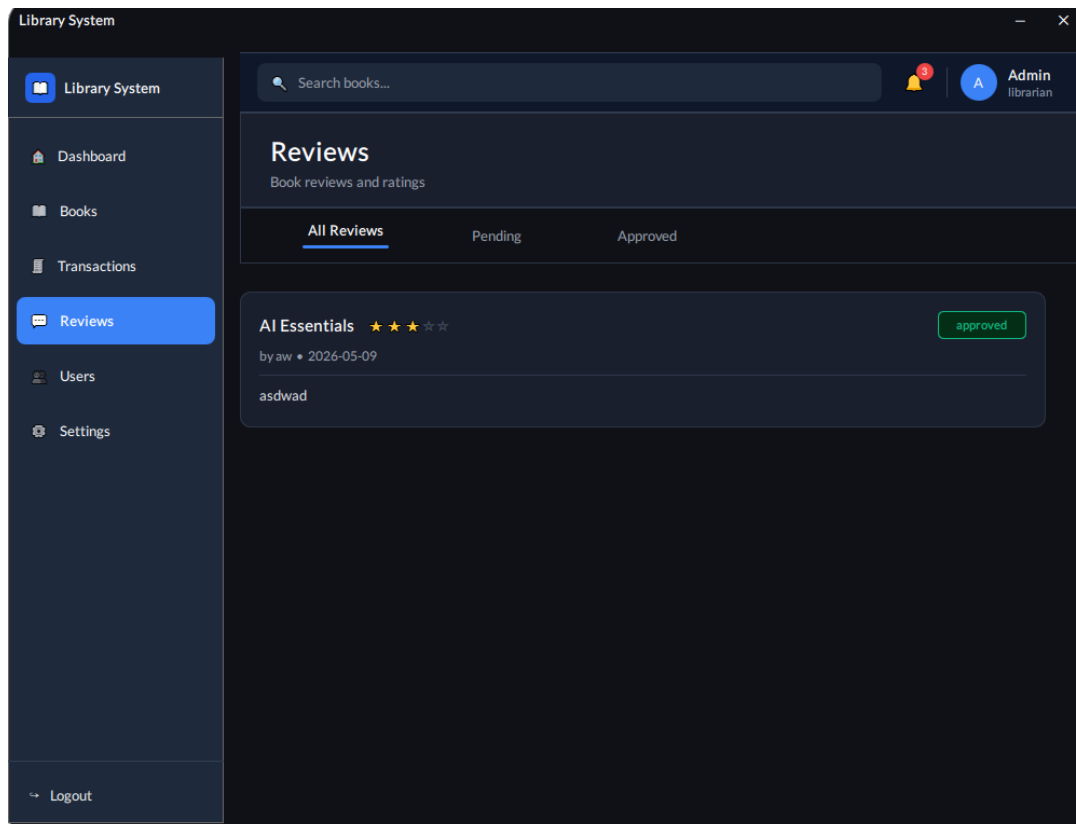
(Figure 4.2: Student Dashboard)



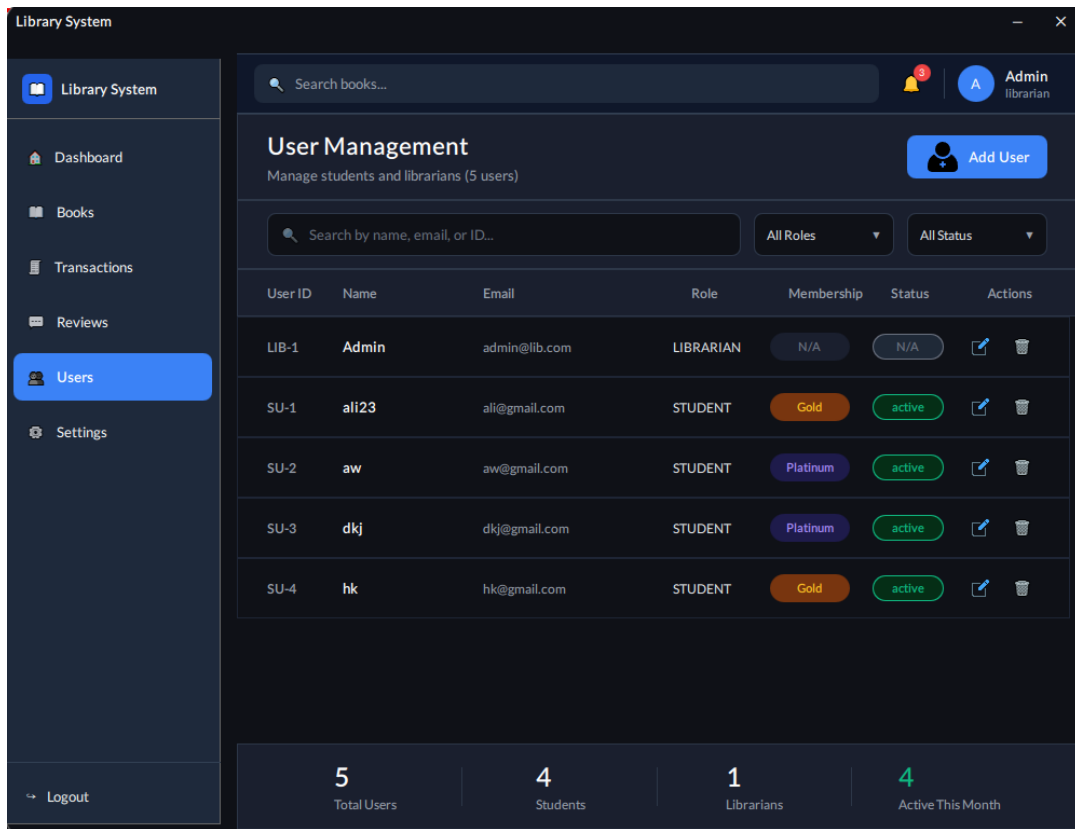
(Figure 4.3: Admin Dashboard)



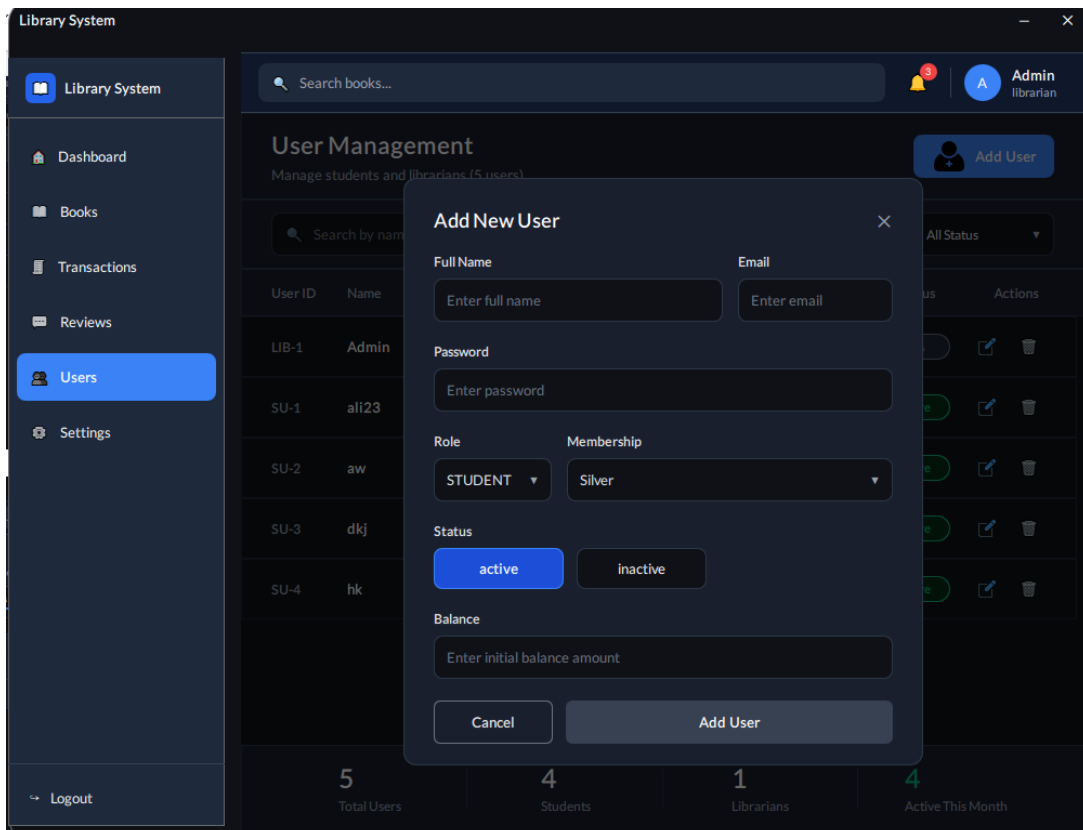
(Figure 4.4: Book Catalog)



(Figure 4.5: Reviews Log)



(Figure 4.6: User Management Screen)



(Figure 4.7: New User Entry)

Library System

Library System

Search books...

Admin librarian

Profile Settings

Manage your account information

Full Name

Admin

Email

admin@lib.com

Role

Librarian

Save Changes

Change Picture

Change Password

Current Password

Enter current password

New Password

Min. 8 characters

Logout

(Figure 4.8: Admin Profile Settings)

Library System

Library System

Search books...

Admin librarian

Transactions

Manage book borrowing and returns (7 transactions)

Issue Book

Return Book

Search by transaction ID, student, or book...

All Status

Txn ID	Student	Book	Issued	Due	Fine	Status
TX-1	ali23 SU-1	C++ 978-1	2026-05-08	2026-05-23	--	active
TX-2	ali23 SU-1	Python 978-2	2026-05-08	2026-05-23	--	active
TX-3	aw SU-2	Python 978-2	2026-05-09	2026-06-08	--	returned
TX-4	aw SU-2	future Web development 981-32	2026-05-09	2026-06-08	--	returned
TX-5	hk SU-4	AI Essentials 978-3	2026-05-09	2026-05-23	--	returned

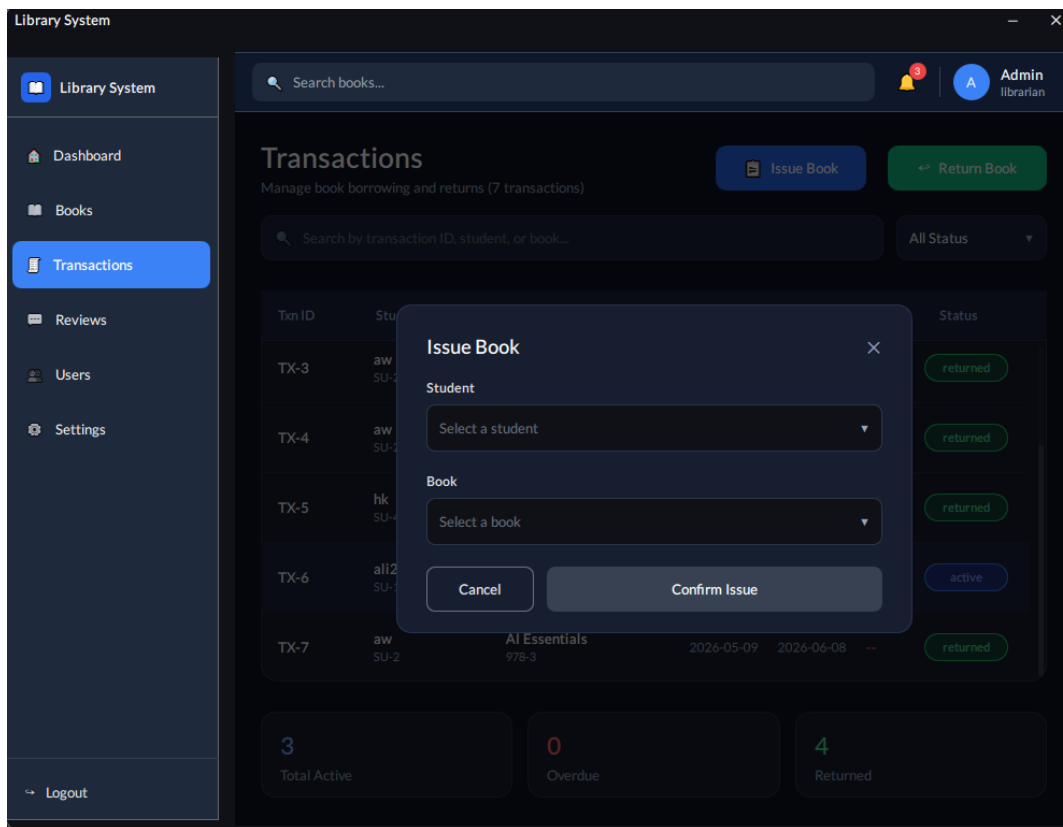
3
Total Active

0
Overdue

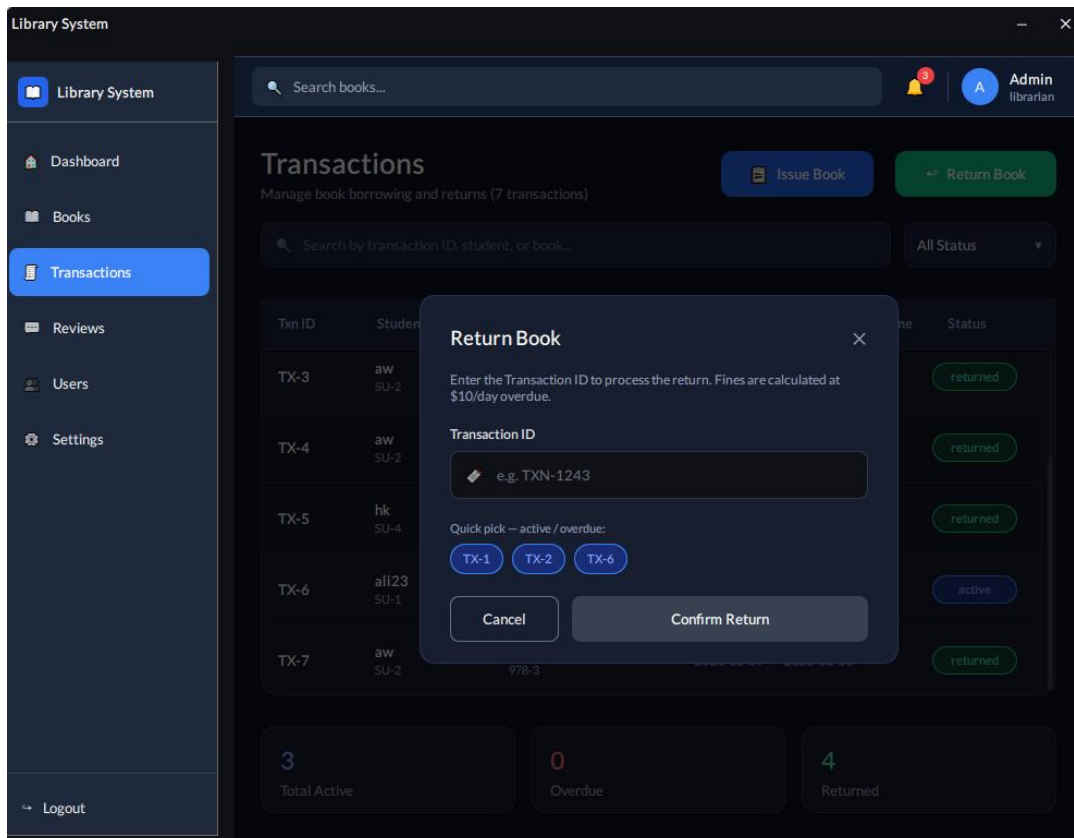
4
Returned

Logout

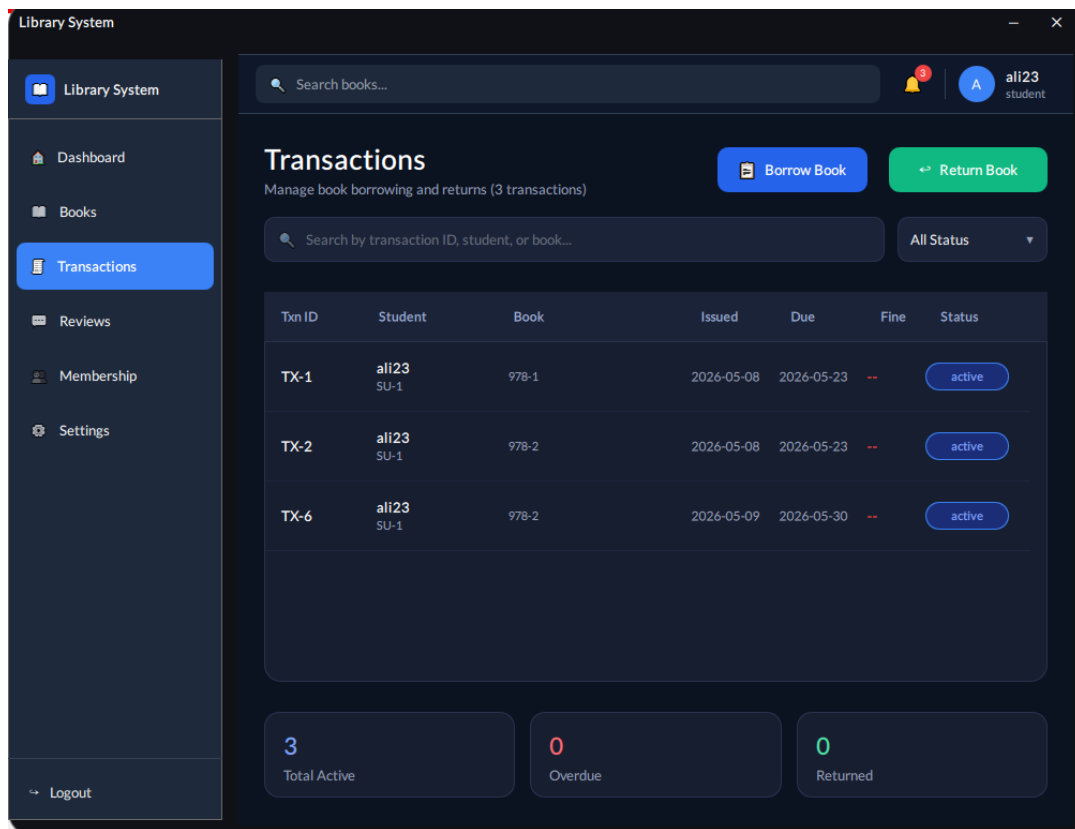
(Figure 4.9: Admin Transaction View)



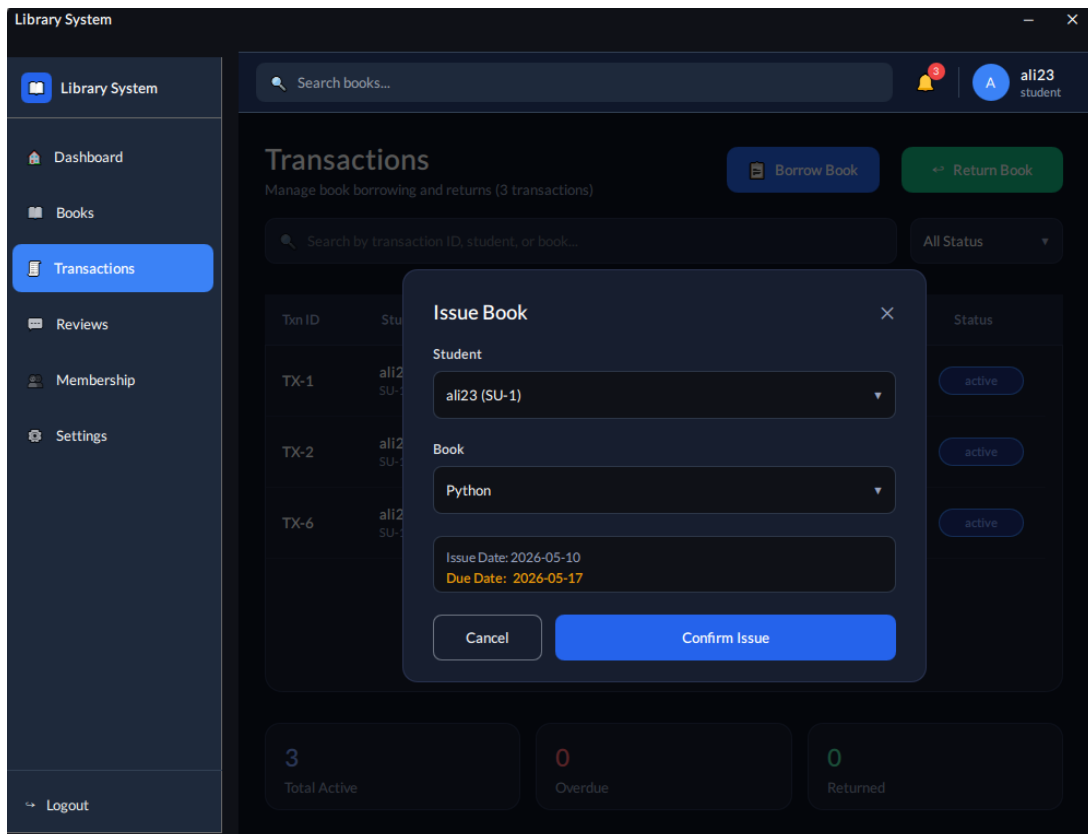
(Figure 4.10: Admin Issue Book View)



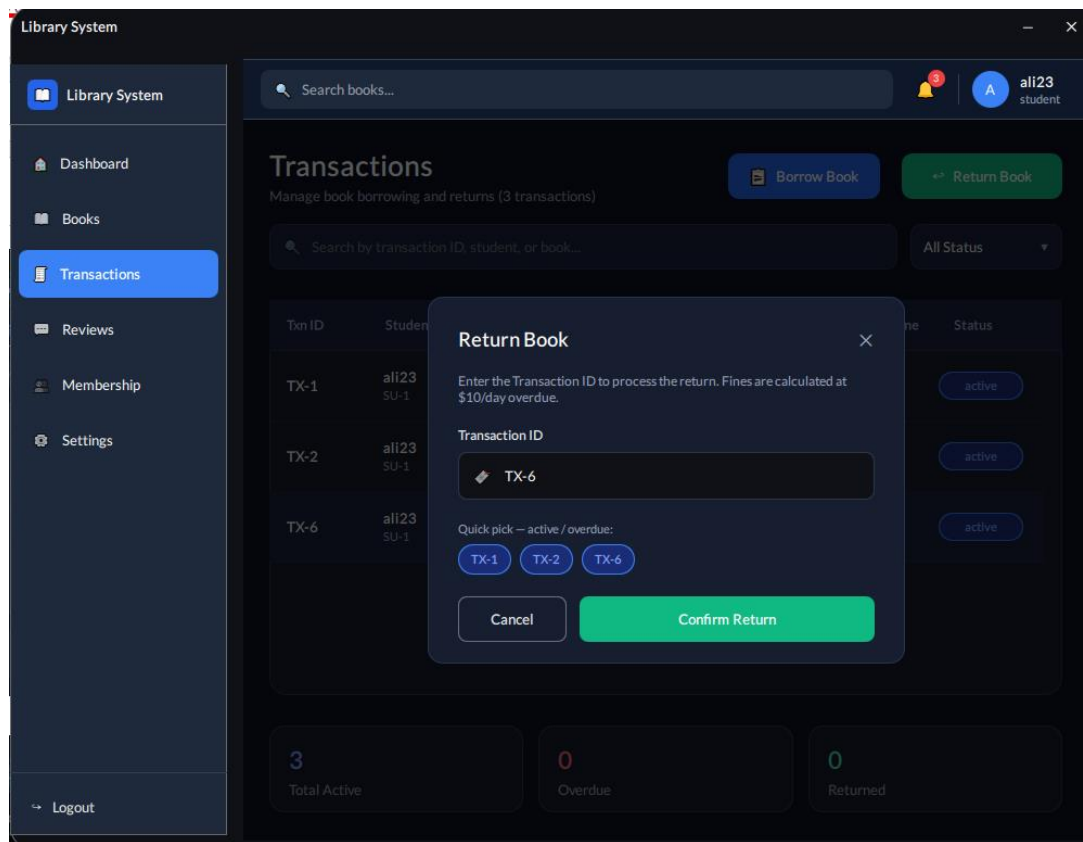
(Figure 4.11: Admin Return Book View)



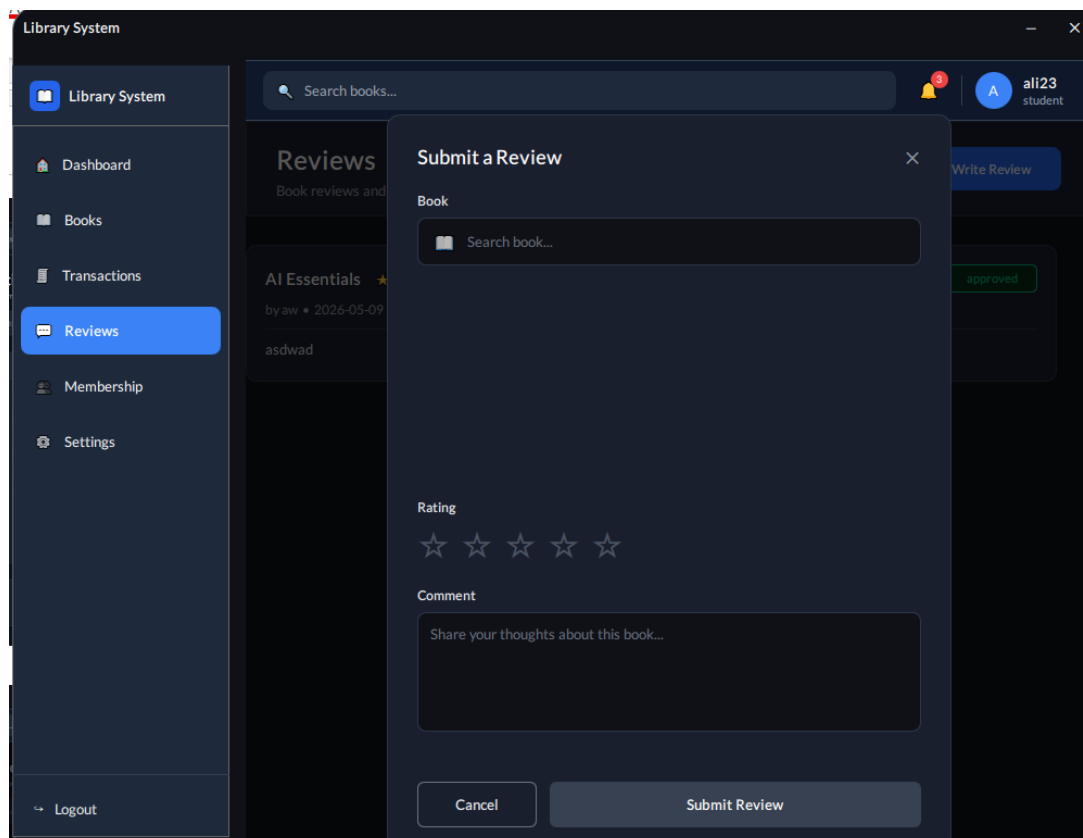
(Figure 4.12: Student Transaction View)



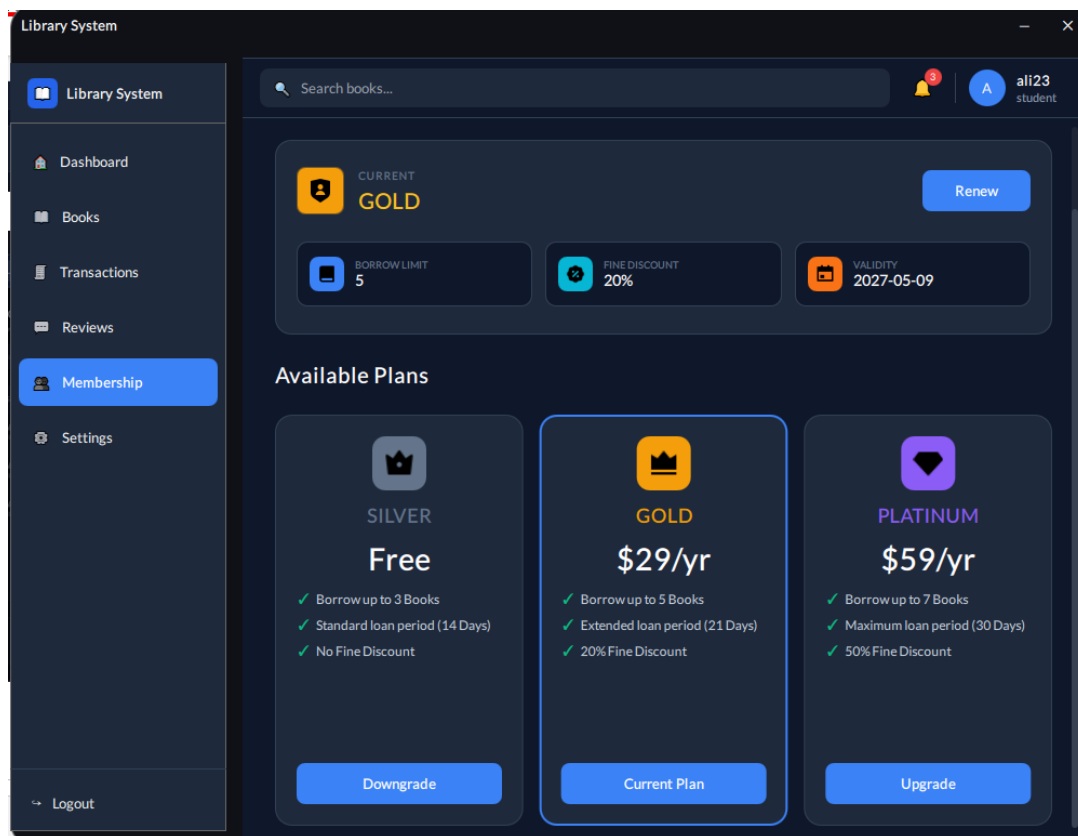
(Figure 4.13: Student Issue Book View)



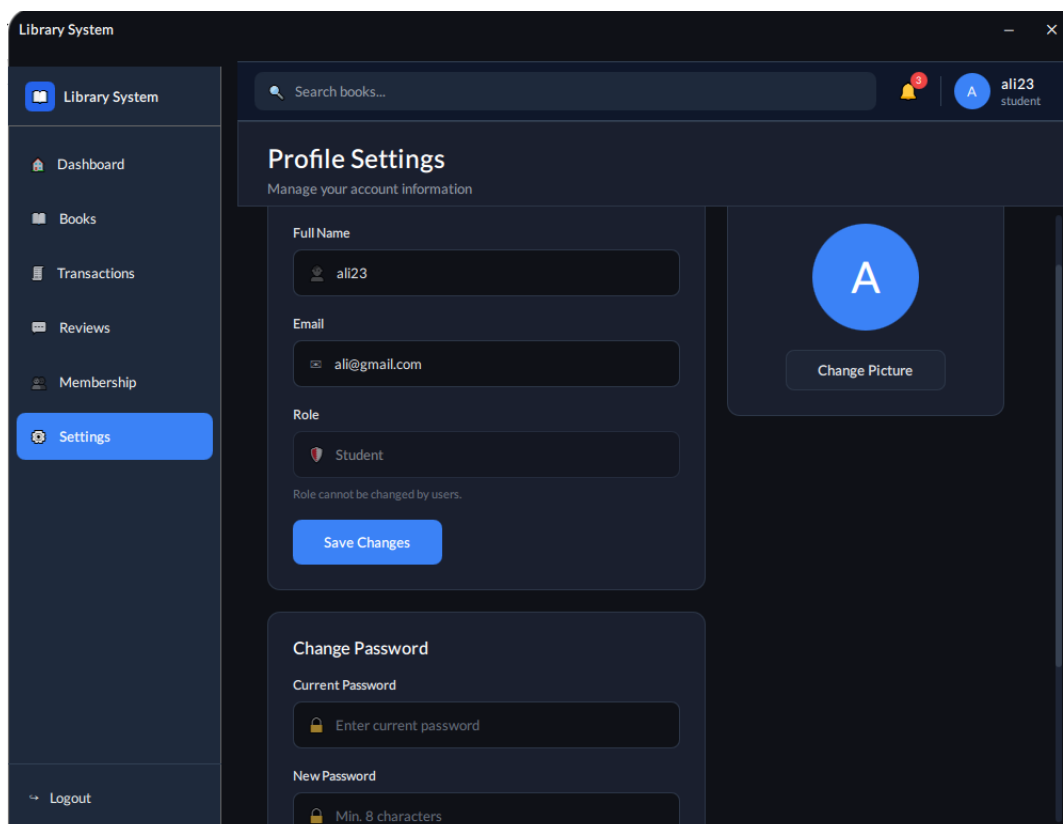
(Figure 4.14: Student Return Book View)



(Figure 4.15: Review Submission)



(Figure 4.16: Student Membership Status)



(Figure 4.17: Student Profile Settings)